
Security Review Report

NM-0828 - Particle CS



NETHERMIND
SECURITY

(May 28, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Component layering	4
4.1.1	EngineBlox	4
4.1.2	BaseStateMachine	4
4.1.3	Components	4
4.1.4	Concrete deployable contracts	4
4.2	Transaction lifecycle	5
4.2.1	Time-delay flow	5
4.2.2	Meta-transaction flow	5
4.2.3	External execution	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[Low] Removing a target from the whitelist can cause transactions in the PENDING state to become stuck	7
6.2	[Low] Target whitelist is checked after the external call has executed	8
6.3	[Info] In-memory writes to record.status and record.result in executeTransaction(...) have no effect	9
6.4	[Info] validateDeadline(...) and validateMetaTxDeadline(...) behave differently when block.timestamp == deadline	10
6.5	[Info] validateTransactionExists(...) does not bound txId against txCounter	10
7	Documentation Evaluation	11
8	Test Suite Evaluation	12
8.1	Tests Output	12
9	About Nethermind	32

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [ParticleCS](#)'s Blochain protocol contracts which is an enterprise-grade security framework for building smart contracts.

The core thesis is that a single compromised key should never be sufficient to immediately do damage. The protocol enforces this architecturally through two parallel workflows that integrators can mix freely per operation:

- **Time-delay workflow** — two on-chain transactions over a configured time window (request → wait → approve). The window provides an intervention opportunity for the rightful party to cancel.
- **Meta-transaction workflow** — one on-chain transaction with two distinct roles: the signer produces an EIP-712 signature off-chain, and the executor submits it on-chain. The signer/executor split is enforced architecturally via the `ConflictingMetaTxPermissions` invariant — a single role cannot hold both `SIGN_META_*` and `EXECUTE_META_*` permissions for the same selector.

The reference identity model uses three **protected roles** that are bootstrapped at deployment and cannot be removed: **Owner**, **Broadcaster**, and **Recovery**. On top of these, integrators may use the **RuntimeRBAC** component to define and manage arbitrary additional roles dynamically.

The audit comprises 3800 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report five points of attention, where two are classified as Low and and three are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

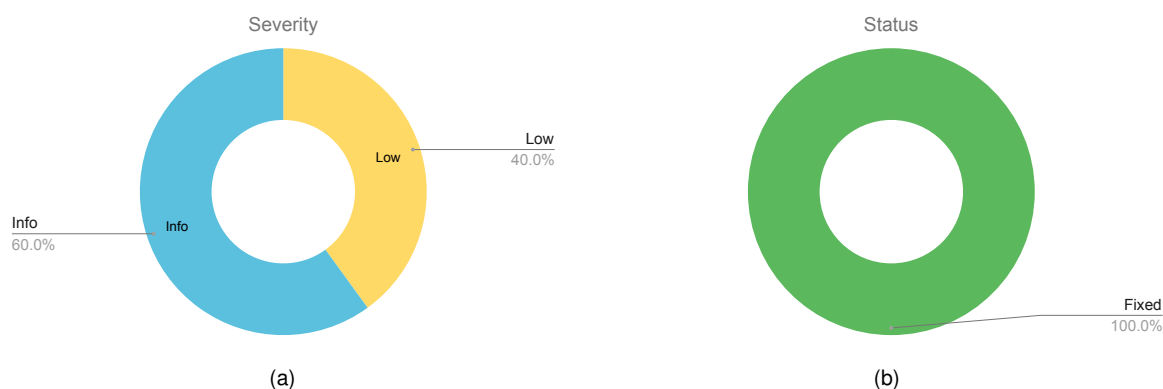


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (2), Undetermined (0), Informational (3), Best Practices (0).
Distribution of status: Fixed (5), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	May 15, 2026
Final Report	May 28, 2026
Initial Commit	8bbef3ab46d4bc4c97a70746ab365d3a9c3b453c
Final Commit	cf91f1a2b9918571d81c89c06ec12091e98af03a
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	access/RuntimeRBAC.sol	112	123	109%	21	255
2	access/interface/IRuntimeRBAC.sol	19	30	157%	6	55
3	access/lib/definitions/RuntimeRBACDefinitions.sol	147	99	67%	44	290
4	security/SecureOwnable.sol	195	215	110%	46	456
5	security/interface/ISecureOwnable.sol	16	72	450%	17	105
6	security/lib/definitions/SecureOwnableDefinitions.sol	543	149	27%	110	802
7	execution/GuardController.sol	203	209	103%	43	454
8	execution/interface/IGuardController.sol	53	81	152%	12	146
9	execution/lib/definitions/GuardControllerDefinitions.sol	372	142	38%	75	589
10	lib/EngineBlox.sol	1396	999	71%	310	2683
11	lib/utls/SharedValidation.sol	234	241	103%	65	540
12	lib/interfaces/IEventForwarder.sol	12	19	158%	2	33
13	lib/interfaces/IDefinition.sol	12	32	266%	5	49
14	pattern/Account.sol	34	44	129%	6	84
15	base/BaseStateMachine.sol	414	462	111%	94	967
16	base/interface/IBaseStateMachine.sol	38	95	250%	20	153
	Total	3800	3012	79.3%	876	7661

3 Summary of Issues

	Finding	Severity	Update
1	Removing a target from the whitelist can cause transactions in the PENDING state to become stuck	Low	Fixed
2	Target whitelist is checked after the external call has executed	Low	Fixed
3	In-memory writes to record.status and record.result in executeTransaction(...) have no effect	Info	Fixed
4	validateDeadline(...) and validateMetaTxDeadline(...) behave differently when block.timestamp == deadline	Info	Fixed
5	validateTransactionExists(...) does not bound txId against txCounter	Info	Fixed

4 System Overview

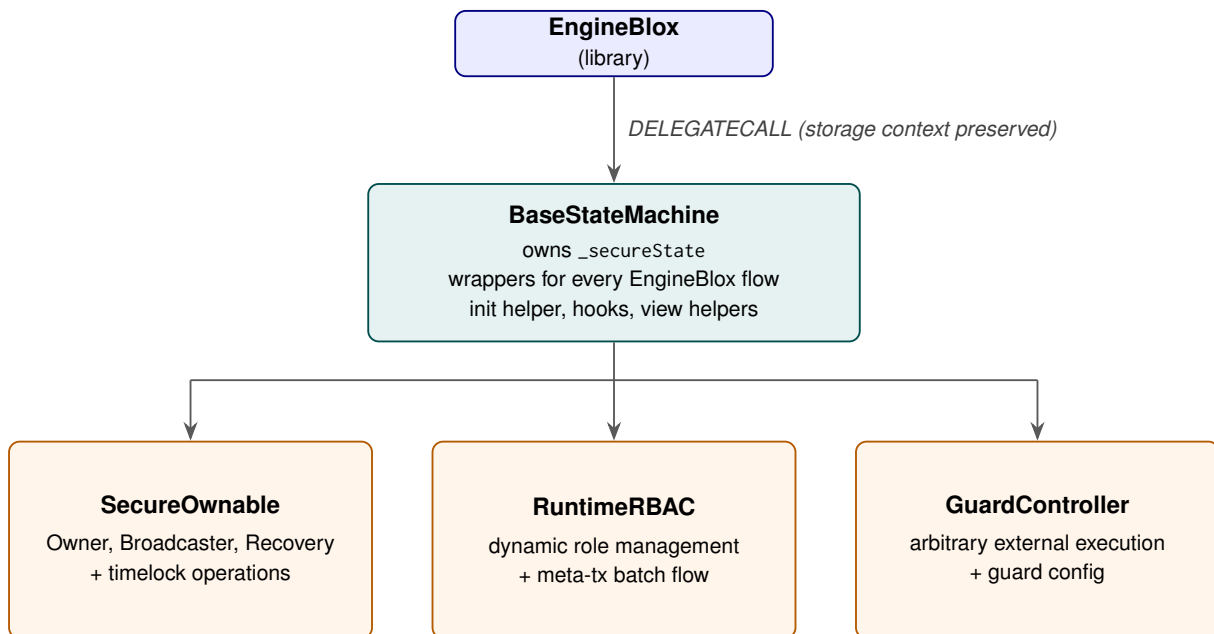
The design rests on three principles that together define the protocol's security posture: a single source of mutation, mandatory two-signature authorization, and defense in depth via redundant gates.

Single source of mutation. All mutable state lives in one struct (`SecureOperationState`) instantiated once per deployed contract. Only the EngineBlox library may mutate it, which isolates the protocol's invariants to a single audit surface.

Two-signature mandatory. Every state-changing operation must be authorized by at least two distinct parties — either across time (request now, approve later, with a window to intervene) or across roles (signer $\neq$ executor for meta-tx). This is enforced architecturally rather than by convention.

Defense in depth via redundant gates. Where one check could be sufficient, the protocol layers two or more checks that look at the same property from different angles: identity vs. role membership, permission grant vs. handler/execution wiring, and storage status vs. structural invariants. A single compromised layer cannot bypass the others.

4.1 Component layering



The framework is organized as three tiers — a stateless library at the top, a single base contract that owns the protocol's storage, and three composable components that each contribute a slice of the public surface. Sections 4.1.1 through 4.1.3 describe each tier, and Section 4.1.4 explains how a concrete deployable contract is assembled from them.

4.1.1 EngineBlox

EngineBlox is a Solidity library, it owns no storage of its own. Its public functions are linked into integrating contracts via `DELEGATECALL`, so when EngineBlox runs, `address(this)` and storage context belong to the integrating contract. The library mutates whatever `SecureOperationState` storage reference the caller passes in.

4.1.2 BaseStateMachine

`BaseStateMachine` is the only contract that declares `_secureState`. All component contracts inherit from it, directly or transitively. This guarantees a *single* `_secureState` slot per deployed contract — regardless of how many components are composed.

4.1.3 Components

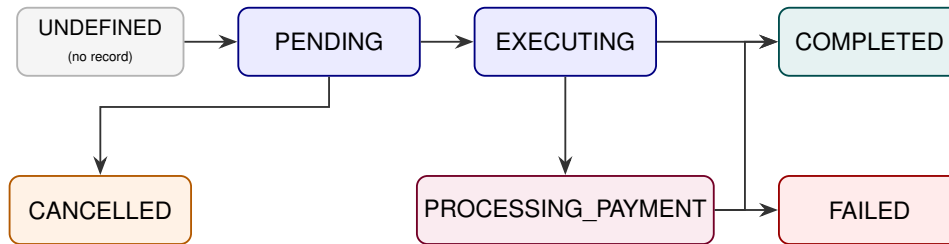
Each component (`SecureOwnable`, `RuntimeRBAC`, `GuardController`) adds its own public surface — the user-facing operations for ownership transfer, role config, and external execution — plus its own definitions library that declares its function schemas and role permission grants.

4.1.4 Concrete deployable contracts

A concrete deployable contract inherits whichever components it needs. A wallet-style contract typically composes all three. A vault, an ERC20, or a payment scheduler may inherit only `SecureOwnable`. A clone factory may inherit only `BaseStateMachine`. The composition is per-deployment; the framework does not prescribe a single pattern.

4.2 Transaction lifecycle

Every operation in the protocol is represented as a TxRecord indexed by a monotonically-increasing txId, and each record carries a single TxStatus enum value that tracks its progression through the lifecycle:



The protocol exposes two parallel workflow patterns that integrators may mix per operation. Both perform the same PENDING → EXECUTING → terminal state (COMPLETED, FAILED, or CANCELLED) progression; they differ in how authorization is supplied and whether a wait window is enforced.

4.2.1 Time-delay flow

This pattern uses two on-chain transactions over a configured window. An initiator with EXECUTE_TIME_DELAY_REQUEST permission creates a PENDING record with `releaseTime = block.timestamp + timeLockPeriodSec`. After the window elapses, an approver with EXECUTE_TIME_DELAY_APPROVE permission moves the record through EXECUTING to a terminal state (COMPLETED, FAILED, or CANCELLED). A separate cancel path with EXECUTE_TIME_DELAY_CANCEL permission can short-circuit to CANCELLED at any time during the window.

4.2.2 Meta-transaction flow

This pattern uses one on-chain transaction with two distinct roles. A signer with SIGN_META_* permission produces an EIP-712 typed-data signature off-chain over the full MetaTransaction struct. An executor with EXECUTE_META_* permission then submits the signed message on-chain. EngineBlox runs a full integrity battery (signature length, chainId, deadline, gasprice, nonce, handlerContract bind, signer-permission dual check, ECDSA recovery) before incrementing the signer's nonce and moving the record through EXECUTING to a terminal state within the same call.

A fused “request-and-approve” meta-tx variant is used for operations the protocol designers chose not to gate with a time-delay — recovery rotation, time-lock change, role-config batch, and guard-config batch. The two-signature property is preserved, but the time-lock observation window is omitted.

4.2.3 External execution

The EXECUTING transition is the only moment the protocol calls arbitrary external code. Returndata copying is bounded to 32 KiB, defending against returnbomb griefing. The target address must clear a per-function target whitelist (`functionTargetWhitelist[executionSelector]`); an empty whitelist means deny, with no implicit wildcard. A separate system macro selector set permits `target == address(this)` only for specific intentionally-self-call execution selectors (e.g., `executeTransferOwnership`). The whitelist is checked at request time and re-checked before payment-leg external calls inside `executeAttachedPayment`.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Removing a target from the whitelist can cause transactions in the PENDING state to become stuck

File(s): [contracts/core/lib/EngineBlox.sol](#)

Description: A pending transaction in EngineBlox can leave the PENDING status only through the approval paths or through the cancellation paths. Both ways verifies that the target for the transaction is whitelisted:

```
1 function _cancelTransaction(SecureOperationState storage self, uint256 txId) private {
2     // @audit-issue This call blocks cancellation when the target is no longer whitelisted.
3     _validateTargetWhitelist(
4         self,
5         self.txRecords[txId].params.executionSelector,
6         self.txRecords[txId].params.target
7     );
8
9     self.txRecords[txId].status = TxStatus.CANCELLED;
10
11     removePendingTx(self, txId);
12
13     logTxEvent(self, txId, self.txRecords[txId].params.executionSelector);
14 }
```

This can cause a scenario where a transaction is stuck in PENDING state:

1. A requester calls `txRequest(...)` with target T, while T is whitelisted for `executionSelector`. The record is stored with `status = PENDING`;
2. The whitelist administrator removes T from `self.functionTargetWhitelist[executionSelector]`;
3. Any attempt to call `txDelayedApproval(...)` or `_txApprovalWithMetaTx(...)` reverts inside `_completeTransaction(...)` on the whitelist check, after consuming gas for the external call;
4. Any attempt to call `txCancellation(...)` or `txCancellationWithMetaTx(...)` reverts inside `_cancelTransaction(...)` on the same whitelist check, before any state transition can occur;

The record is stuck in `TxStatus.PENDING` indefinitely. The only way to unblock it is for an administrator to re-add T to the whitelist.

Recommendation(s): Consider removing the call to `_validateTargetWhitelist(...)` from `_cancelTransaction(...)`. Cancellation does not interact with the target and does not depend on the target being currently authorized.

Status: Fixed.

Update from the client: Removed `_validateTargetWhitelist(...)` from `_cancelTransaction(...)`. Fixed in commit [72d67d](#).

6.2 [Low] Target whitelist is checked after the external call has executed

File(s): `contracts/core/lib/EngineBlox.sol`

Description: Target whitelisting is enforced through `_validateTargetWhitelist(...)`, which checks that target is registered for `executionSelector` in `self.functionTargetWhitelist`. The goal is to restrict every approved transaction to a destination that was authorized before.

When `txDelayedApproval(...)` is called, the functions perform the external call first and after that they call `_completeTransaction(...)`, which is where the whitelist is checked:

```

1  function txDelayedApproval(SecureOperationState storage self, uint256 txId, bytes4 handlerSelector)
2      public
3      returns (TxRecord memory)
4  {
5      // ...
6      self.txRecords[txId].status = TxStatus.EXECUTING;
7
8      // INTERACT: External call after state update
9      (bool success, bytes memory result) = executeTransaction(self, self.txRecords[txId]);
10
11     // @audit-issue Whitelist is enforced only after the external call has run.
12     _completeTransaction(self, txId, success, result);
13     // ...
14 }

```

```

1  function _completeTransaction(
2      SecureOperationState storage self,
3      uint256 txId,
4      bool success,
5      bytes memory result
6  ) private {
7      // @audit-issue This check belongs before `executeTransaction(...)` , not after it.
8      _validateTargetWhitelist(
9          self,
10         self.txRecords[txId].params.executionSelector,
11         self.txRecords[txId].params.target
12     );
13     // ...
14 }

```

The same pattern repeats in `_txApprovalWithMetaTx(...)`, which also calls `executeTransaction(...)` before `_completeTransaction(...)`.

Two consequences follow from this ordering.

First, gas is wasted on the unhappy path. If the target is no longer whitelisted at approval time, for example because an admin removed it between request and approval, the engine still flips status to `EXECUTING`, forwards the configured gas to the target, and lets the target execute arbitrary code before `_completeTransaction(...)` reverts the entire transaction on the late whitelist check. The caller pays for all of that work even though the call should never have been issued.

Second, the late check breaks checks-effects-interactions pattern.

Recommendation(s): Consider moving `_validateTargetWhitelist(...)` so that the whitelist is enforced before any external call is made.

Status: Fixed.

Update from the client: The finding was valid as a gas-efficiency and CEI-ordering issue. `_validateTargetWhitelist(...)` was moved out of `_completeTransaction(...)` and is enforced before any external call on every approval path (`txDelayedApproval(...)` and `_txApprovalWithMetaTx(...)`). Fixed in commit [72d67d](#).

6.3 [Info] In-memory writes to record.status and record.result in executeTransaction(...) have no effect

File(s): [contracts/core/lib/EngineBlox.sol](#)

Description: The executeTransaction(...) helper performs the low-level call associated with an approved transaction. The helper accepts the transaction record as TxRecord memory record, which is a memory copy of self.txRecords[txId] taken at the call site.

After performing the external call, the helper transitions the record status and store the returned data depending on the call outcome:

```

1  function executeTransaction(SecureOperationState storage self, TxRecord memory record)
2      private
3      returns (bool, bytes memory)
4  {
5      // ...
6      if (success) {
7          // @audit-issue Writing to a memory copy has no effect on the persisted record.
8          record.status = TxStatus.COMPLETED;
9          record.result = result;
10
11          if (record.payment.recipient != address(0)) {
12              executeAttachedPayment(self, record);
13          }
14      } else {
15          // @audit-issue Writing to a memory copy has no effect on the persisted record.
16          record.status = TxStatus.FAILED;
17          record.result = result;
18      }
19
20      return (success, result);
21  }
```

Because record is a memory parameter rather than a storage reference, both branches mutate a local copy that is discarded once the function returns. The persisted status and result are written only later by _completeTransaction(...), which the callers invoke after executeTransaction(...) returns. The intermediate memory writes are dead code. They are not consumed by the caller, the nested executeAttachedPayment(...) re-reads status from storage via _validateTxStatus(self, record.txId, TxStatus.EXECUTING) instead of from record.status, and any external reader of self.txRecords[txId] observes the values set by _completeTransaction(...).

Recommendation(s): Consider removing the assignments to record.status and record.result from executeTransaction(...), since they have no effect.

Status: Fixed.

Update from the client: Removed dead record.status and record.result assignments. Fixed in commit [72d67d](#).

6.4 [Info] validateDeadline(...) and validateMetaTxDeadline(...) behave differently when block.timestamp == deadline

File(s): [contracts/core/lib/utils/SharedValidation.sol](#), [contracts/core/lib/EngineBlox.sol](#)

Description: The protocol uses two different helpers to decide whether the deadline field of a meta-transaction is still valid. They are intended to express the same predicate but they treat the case `block.timestamp == deadline` differently.

```
1 function validateDeadline(uint256 deadline) internal view {
2     // @audit-issue Strict comparison rejects `block.timestamp == deadline`.
3     if (deadline <= block.timestamp) revert DeadlineInPast(deadline, block.timestamp);
4 }
```

```
1 function validateMetaTxDeadline(uint256 deadline) internal view {
2     // @audit-issue Non-strict comparison accepts `block.timestamp == deadline`.
3     if (block.timestamp > deadline) revert MetaTxExpired(deadline, block.timestamp);
4 }
```

`validateDeadline(...)` is used in `generateMetaTransaction(...)`, which is the helper that builds the `MetaTransaction` payload that an off-chain signer is expected to sign. `validateMetaTxDeadline(...)` is invoked from the meta-transaction verification path that runs inside `verifySignature(...)` and is reached from `txApprovalWithMetaTx(...)`, `txCancellationWithMetaTx(...)`, and the request-and-approve meta-tx variants.

This produces an inconsistency at the exact second equal to deadline. A meta-transaction whose deadline matches the current block timestamp at verification time is still executed normally, because `validateMetaTxDeadline(...)` admits the boundary. The same deadline, however, cannot be used to construct a fresh `MetaTransaction` payload through `generateMetaTransaction(...)`, because `validateDeadline(...)` rejects it.

Recommendation(s): Consider aligning the two helpers on the same boundary semantics.

Status: Fixed.

Update from the client: Removed `validateDeadline(...)` and unified on `validateMetaTxDeadline(...)`. Fixed in commit [72d67d](#).

6.5 [Info] validateTransactionExists(...) does not bound txId against txCounter

File(s): [contracts/core/lib/utils/SharedValidation.sol](#), [contracts/core/lib/EngineBlox.sol](#)

Description: The `validateTransactionExists(...)` helper in `SharedValidation` is named so that callers can rely on it to confirm that a given `txId` corresponds to a real transaction record. Its current implementation only rejects the zero value:

```
1 function validateTransactionExists(uint256 txId) internal pure {
2     if (txId == 0) revert ResourceNotFound(bytes32(uint256(txId)));
3 }
```

In `EngineBlox`, transaction identifiers are minted sequentially. Each new record receives `txId = self.txCounter + 1` inside `createTxRecord(...)`, and `self.txCounter` is then incremented in `txRequest(...)`, so any allocated identifier lies in the inclusive range `[1, self.txCounter]`. Identifiers strictly greater than `self.txCounter` are not valid attached to any existing record, however the function accepts them as if they did.

Recommendation(s): Consider modifying `validateTransactionExists(...)` so that it also rejects any `txId` greater than the current `self.txCounter`.

Status: Fixed.

Update from the client: Added `txCounter` parameter and reject `txId > txCounter`. Fixed in commit [72d67d](#).

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- **Technical whitepaper:** A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- **User manual:** A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- **Code documentation:** Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- **API documentation:** API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- **Testing documentation:** Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- **Audit documentation:** Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Particle CS Bloxchain protocol documentation

The Particle CS team provided documentation detailing the full Bloxchain protocol architecture. Additionally, the team provided a walkthrough of the entire protocol including all the main flows with expected state changes. Furthermore, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
> npx run test:foundry

> Bloxchain@1.0.0-alpha.20 test:foundry
> forge test

No files changed, compilation skipped

Ran 5 tests for test/foundry/security/AccessControl.t.sol:AccessControlTest
[PASS] test_GuardConfigBatchExecutionParams() (gas: 11196)
[PASS] test_PermissionBoundary_OwnerOnly() (gas: 25869)
[PASS] test_Revert_ProtectedRoleModification() (gas: 30800)
[PASS] test_Revert_UnauthorizedOwnershipTransfer() (gas: 24038)
[PASS] test_Revert_UnauthorizedRoleCreation() (gas: 14474)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 9.58ms (252.75µs CPU time)

Ran 3 tests for test/foundry/integration/MetaTransaction.t.sol:MetaTransactionTest
[PASS] test_MetaTransaction_CreateParams() (gas: 15864)
[PASS] test_MetaTransaction_MessageHashConsistency() (gas: 503467)
[PASS] test_MetaTransaction_OwnershipTransfer_CompleteWorkflow() (gas: 490971)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 10.11ms (1.48ms CPU time)

Ran 10 tests for test/foundry/unit/RuntimeRBAC.t.sol:RuntimeRBACTest
[PASS] test_CannotModifyProtectedRoles() (gas: 47583)
[PASS] test_ExecuteRoleConfigBatch_AddWallet() (gas: 13756)
[PASS] test_ExecuteRoleConfigBatch_CreateRole() (gas: 8031)
[PASS] test_GetFunctionSchema_RegisteredFunction() (gas: 8361)
[PASS] test_GetFunctionSchema_Revert_UnregisteredFunction() (gas: 23836)
[PASS] test_GetWalletsInRole_RequiresRole() (gas: 30480)
[PASS] test_GetWalletsInRole_Revert_NoRole() (gas: 15893)
[PASS] test_GetWalletsInRole_Revert_RoleNotFound() (gas: 18234)
[PASS] test_SupportsInterface_ERC165() (gas: 6055)
[PASS] test_SupportsInterface_IRuntimeRBAC() (gas: 6123)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 10.46ms (1.05ms CPU time)

Ran 2 tests for test/foundry/unit/PaymentRecipientWhitelist.t.sol:PaymentRecipientWhitelistTest
[PASS] test_RevertWhen_PaymentRecipientNotWhitelisted_selfTarget_native() (gas: 50565)
[PASS] test_SucceedsWhen_PaymentRecipientWhitelisted() (gas: 447437)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.17ms (115.67µs CPU time)

Ran 6 tests for test/foundry/fuzz/SystemMacroSelectorSecurityFuzz.t.sol:SystemMacroSelectorSecurityFuzzTest
[PASS] testFuzz_NonMacroSelectorsCannotTargetAddressThis(bytes4) (runs: 256, : 30230, ~: 30230)
[PASS] testFuzz_SystemMacroSelectorIdentification(bytes4) (runs: 256, : 17124, ~: 17124)
[PASS] testFuzz_SystemMacroSelectorsCanTargetAddressThis(uint256) (runs: 256, : 483668, ~: 483426)
[PASS] testFuzz_SystemMacroSelectorsRequirePermissions(address,uint256) (runs: 256, : 34350, ~: 34123)
[PASS] testFuzz_SystemMacroSelectorsRespectWhitelist(address,uint256) (runs: 256, : 503825, ~: 503594)
[PASS] testFuzz_UpdatePaymentSelectorRequiresPermissions(address,uint256) (runs: 256, : 28991, ~: 28764)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 200.56ms (196.14ms CPU time)

Ran 1 test for test/foundry/helpers/TestStateMachine.sol:TestStateMachine
[PASS] testFunction() (gas: 667)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 814.29µs (14.08µs CPU time)

Ran 20 tests for test/foundry/fuzz/ComprehensiveDefinitionSecurityFuzz.t.sol:ComprehensiveDefinitionSecurityFuzzTest
[PASS] testFuzz_AllProtectedSchemasSucceedWhenEnforced(bytes4) (runs: 256, : 315403, ~: 315403)
[PASS] testFuzz_DefinitionWithEmptyBitmapRejected(bytes4) (runs: 256, : 339328, ~: 339328)
[PASS] testFuzz_DefinitionWithInvalidHandlerSelectorsRejected(bytes4,bytes4) (runs: 256, : 322519, ~: 322519)
[PASS] testFuzz_DefinitionWithMismatchedPermissionArraysRejected(uint256,uint256) (runs: 256, : 333643, ~: 333680)
[PASS] testFuzz_DefinitionWithSelfReferenceBehavior(bytes4) (runs: 256, : 341138, ~: 341138)
[PASS] testFuzz_MixedSchemasRejectedWhenEnforced(bytes4,bytes4) (runs: 256, : 23400, ~: 23400)
[PASS] testFuzz_MultipleDefinitionLoadingHandled() (gas: 587955)
[PASS] testFuzz_NonProtectedSchemaRejectedWhenEnforced(bytes4) (runs: 256, : 18379, ~: 18379)
[PASS] testFuzz_NonProtectedSchemasWorkWhenNotEnforced(bytes4) (runs: 256, : 315237, ~: 315237)
[PASS] testFuzz_SchemaRegistrationOrderEnforced() (gas: 478072)
[PASS] testStateMachine() (gas: 3267)
[PASS] test_DefinitionValidationPreventsAllAttackVectors() (gas: 490)
[PASS] test_DefinitionWithDuplicateSchemasRejected() (gas: 328655)
[PASS] test_DefinitionWithEmptyHandlerArrayRejected() (gas: 22915)
[PASS] test_DefinitionWithMismatchedSignatureRejected() (gas: 23887)
```

```
[PASS] test_DefinitionWithMissingProtectedFlagRejected() (gas: 25306)
[PASS] test_PermissionForNonExistentFunctionRejected() (gas: 34461)
[PASS] test_SystemDefinitionContractsValid() (gas: 312532)
[PASS] test_SystemDefinitionsProtectSystemFunctions() (gas: 20275538)
[PASS] test_ValidDefinitionContractsLoadSuccessfully() (gas: 325890)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 233.12ms (229.54ms CPU time)
```

```
Ran 34 tests for test/foundry/unit/SecureOwnable.t.sol:SecureOwnableTest
[PASS] invariant_BroadcasterListSafety() (runs: 256, calls: 3840, reverts: 3312)
```

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	112	112	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	103	103	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	104	104	0
AccountPatternTest	executeBroadcasterUpdate	111	111	0
AccountPatternTest	executeGuardConfigBatch	121	121	0
AccountPatternTest	executeRecoveryUpdate	104	104	0
AccountPatternTest	executeRoleConfigBatch	106	106	0
AccountPatternTest	executeTimeLockUpdate	116	116	0
AccountPatternTest	executeTransferOwnership	108	108	0
AccountPatternTest	executeWithPayment	110	110	0
AccountPatternTest	executeWithTimeLock	114	114	0
AccountPatternTest	guardConfigBatchRequestAndApprove	120	120	0
AccountPatternTest	initialize	109	109	0
AccountPatternTest	requestAndApproveExecution	95	95	0
AccountPatternTest	roleConfigBatchRequestAndApprove	111	111	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	141	141	0
AccountPatternTest	transferOwnershipCancellation	127	127	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	104	104	0
AccountPatternTest	transferOwnershipDelayedApproval	109	109	0
AccountPatternTest	transferOwnershipRequest	95	95	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	116	116	0
AccountPatternTest	updateBroadcasterCancellation	111	111	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	121	121	0
AccountPatternTest	updateBroadcasterDelayedApproval	114	114	0
AccountPatternTest	updateBroadcasterRequest	114	114	0
AccountPatternTest	updateRecoveryRequestAndApprove	107	107	0
AccountPatternTest	updateTimeLockRequestAndApprove	100	100	0
MockERC20	approve	112	2	0
MockERC20	mint	108	4	0
MockERC20	transfer	100	96	0

MockERC20	transferFrom	95	94	0
MockEventForwarder	forwardEvent	111	0	0
MockTarget	execute	111	0	0
MockTarget	executeWithParams	87	0	0

```
[PASS] testFuzz_UpdateBroadcasterRequest_RevokeAtLocation(uint256) (runs: 256, : 849524, ~: 856753)
[PASS] test_ExecuteTransferOwnership_InternalCall() (gas: 495553)
[PASS] test_ExecuteTransferOwnership_Revert_ExternalCall() (gas: 15112)
[PASS] test_Initialize_Revert_DoubleInitialization() (gas: 20220160)
[PASS] test_Initialize_Revert_ZeroOwner() (gas: 4205847)
[PASS] test_Initialize_Revert_ZeroRecovery() (gas: 4203177)
[PASS] test_Initialize_Revert_ZeroTimelock() (gas: 4206220)
[PASS] test_Initialize_WithValidParameters() (gas: 20228185)
[PASS] test_MultiplePendingRequests_OnlyOneOwnership() (gas: 420212)
[PASS] test_OwnershipTransfer_CompleteWorkflow() (gas: 525417)
[PASS] test_SupportsInterface_ERC165() (gas: 6176)
[PASS] test_SupportsInterface_ISecureOwnable() (gas: 5662)
[PASS] test_SupportsInterface_InvalidInterface() (gas: 6443)
[PASS] test_TransferOwnershipApprovalWithMetaTx_Valid() (gas: 490249)
[PASS] test_TransferOwnershipCancellation_RecoveryCanCancel() (gas: 411097)
[PASS] test_TransferOwnershipDelayedApproval_AfterTimelock() (gas: 510273)
[PASS] test_TransferOwnershipDelayedApproval_OwnerCanApprove() (gas: 512344)
[PASS] test_TransferOwnershipDelayedApproval_Revert_BeforeTimelock() (gas: 457101)
[PASS] test_TransferOwnershipRequest_RecoveryCanRequest() (gas: 430236)
[PASS] test_TransferOwnershipRequest_Revert_DuplicateRequest() (gas: 420762)
[PASS] test_TransferOwnershipRequest_Revert_Unauthorized() (gas: 24170)
[PASS] test_UpdateBroadcasterCancellation_OwnerCanCancel() (gas: 409607)
[PASS] test_UpdateBroadcasterDelayedApproval_AfterTimelock() (gas: 530405)
[PASS] test_UpdateBroadcasterRequest_OwnerCanRequest() (gas: 432824)
[PASS] test_UpdateBroadcasterRequest_Revert_Unauthorized() (gas: 24600)
[PASS] test_UpdateBroadcasterRequest_RevokeAtLocation_LastBroadcaster_Reverts() (gas: 484321)
[PASS] test_UpdateBroadcasterRequest_RevokeAtLocation_ZeroAddress() (gas: 842411)
[PASS] test_UpdateRecoveryExecutionParams_SameAddress_Encodes() (gas: 7852)
[PASS] test_UpdateRecoveryExecutionParams_ValidAddress() (gas: 7404)
[PASS] test_UpdateRecoveryExecutionParams_ZeroAddress_Encodes() (gas: 4885)
[PASS] test_UpdateTimelockExecutionParams_SamePeriod_Encodes() (gas: 4794)
[PASS] test_UpdateTimelockExecutionParams_ValidPeriod() (gas: 4301)
[PASS] test_UpdateTimelockExecutionParams_ZeroPeriod_Encodes() (gas: 5207)
Suite result: ok. 34 passed; 0 failed; 0 skipped; finished in 407.89ms (398.99ms CPU time)
```

```
Ran 6 tests for test/foundry/fuzz/MetaTransactionSecurityFuzz.t.sol:MetaTransactionSecurityFuzzTest
[PASS] testFuzz_ExpiredMetaTransactionRejected(uint256) (runs: 256, : 48324, ~: 48346)
[PASS] testFuzz_InvalidSignatureLengthRejected(bytes) (runs: 256, : 651881, ~: 651860)
[PASS] testFuzz_NonceReplayProtection() (gas: 1325040)
[PASS] testFuzz_UnauthorizedSignerRejected(uint256) (runs: 256, : 652576, ~: 652591)
[PASS] testFuzz_WrongChainIdRejected(uint256) (runs: 256, : 47575, ~: 47575)
[PASS] testFuzz_WrongNonceRejected(uint256) (runs: 256, : 672087, ~: 672087)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 440.92ms (432.68ms CPU time)
```

```
Ran 11 tests for test/foundry/security/EdgeCases.t.sol:EdgeCasesTest
[PASS] test_BeforeReleaseTime() (gas: 456516)
[PASS] test_ConcurrentOperations() (gas: 433681)
[PASS] test_CreateMetaTxParams_ShortDeadline() (gas: 15851)
[PASS] test_DuplicateRequests() (gas: 420897)
[PASS] test_EmptyArrays() (gas: 5424)
[PASS] test_InvalidStateTransitions() (gas: 500155)
[PASS] test_InvalidTransactionId() (gas: 35021)
[PASS] test_MaxValueBoundaries() (gas: 4922)
[PASS] test_OwnershipRequest_AllowedWhileBroadcasterPending() (gas: 766239)
[PASS] test_SameAddressUpdate() (gas: 7642)
[PASS] test_ZeroAddressHandling() (gas: 4808)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 3.76ms (754.29µs CPU time)
```

```
Ran 8 tests for test/foundry/fuzz/ComprehensiveWhitelistSchemaFuzz.t.sol:ComprehensiveWhitelistSchemaFuzzTest
[PASS] testFuzz_AddressThisBypassIsIntentional() (gas: 473)
[PASS] testFuzz_DuplicateRoleCreationPrevented() (gas: 1154)
[PASS] testFuzz_EmptyWhitelistDeniesExternalTargets() (gas: 253)
[PASS] testFuzz_HandlerSelectorValidationPreventsBypass() (gas: 1111)
[PASS] testFuzz_OperationTypeCleanupWorksCorrectly() (gas: 561)
[PASS] testFuzz_ProtectedFunctionSchemaCannotBeModified() (gas: 165)
[PASS] testFuzz_UnregisteredSelectorBehavior(address,bytes4,bytes) (runs: 256, : 4831, ~: 4831)
```



```
[PASS] testFuzz_WhitelistRemovalPreventsExecution() (gas: 605)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 18.03ms (10.70ms CPU time)
```

Ran 4 tests for test/foundry/fuzz/ComprehensiveEIP712AndViewFuzz.t.sol:ComprehensiveEIP712AndViewFuzzTest

```
[PASS] testFuzz_DomainSeparatorDeterministic(uint256,uint256,uint256) (runs: 256, : 39632, ~: 39686)
[PASS] testFuzz_EIP712RecoversCorrectSigner() (gas: 81932)
[PASS] testFuzz_ExcessMsgValueRefunded(uint96) (runs: 256, : 20910, ~: 20794)
[PASS] testFuzz_ViewReflectsLatestState(string) (runs: 256, : 833086, ~: 833077)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 276.07ms (272.56ms CPU time)
```

Ran 7 tests for test/foundry/unit/StateAbstraction.t.sol:EngineBloxTest

```
[PASS] test_BroadcasterRole_Constant() (gas: 1600)
[PASS] test_MaxResultPreviewBytes_Constant() (gas: 360)
[PASS] test_NativeTransferSelector_Constant() (gas: 778)
[PASS] test_OwnerRole_Constant() (gas: 1050)
[PASS] test_RecoveryRole_Constant() (gas: 1402)
[PASS] test_TransactionStatus_InvalidTransition() (gas: 441942)
[PASS] test_TransactionStatus_Transitions() (gas: 507681)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 2.99ms (354.67µs CPU time)
```

Ran 4 tests for test/foundry/fuzz/ProtectedResourceFuzz.t.sol:ProtectedResourceFuzzTest

```
[PASS] testFuzz_CannotAddWalletToProtectedRole(address) (runs: 256, : 1877998, ~: 1877986)
[PASS] testFuzz_CannotRemoveProtectedRole() (gas: 1815969)
[PASS] testFuzz_CannotRevokeWalletFromProtectedRole() (gas: 697838)
[PASS] testFuzz_ProtectedRolesUnchangedAfterOperation(string,address) (runs: 256, : 1502844, ~: 1502842)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 760.50ms (757.92ms CPU time)
```

Ran 3 tests for test/foundry/fuzz/RuntimeRBACFuzz.t.sol:RuntimeRBACFuzzTest

```
[PASS] testFuzz_GetFunctionSchema(bytes4) (runs: 256, : 20805, ~: 20620)
[PASS] testFuzz_RoleCreation(uint256,uint256) (runs: 256, : 822423, ~: 822649)
[PASS] testFuzz_WalletAssignment(string,address) (runs: 256, : 1487593, ~: 1487592)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 791.72ms (541.69ms CPU time)
```

Ran 4 tests for test/foundry/invariant/TransactionInvariants.t.sol:TransactionInvariantsTest

```
[PASS] invariant_MetaTransactionNonceMonotonic() (runs: 256, calls: 3840, reverts: 3194)
```

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0

AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_PaymentValidation() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0

AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_ReleaseTimeValidation() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0

MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0
[PASS] invariant_TransactionStatusConsistency() (runs: 256, calls: 3840, reverts: 3194)				
Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0

MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 676.88ms (669.61ms CPU time)

Ran 1 test for test/foundry/fuzz/UnregisterFunctionFuzz.t.sol:UnregisterFunctionFuzzTest
[PASS] test_UnsafeUnregisterPreventsNewRequests() (gas: 2069894)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.79ms (192.08µs CPU time)

Ran 5 tests for test/foundry/integration/WhitelistWorkflow.t.sol:WhitelistWorkflowTest
[PASS] test_Whitelist_ExecutionFailsWithoutWhitelist() (gas: 23571)
[PASS] test_Whitelist_ExecutionParamsCreation() (gas: 12169)
[PASS] test_Whitelist_MultipleTargets() (gas: 12441)
[PASS] test_Whitelist_RemoveExecutionParams() (gas: 11460)
[PASS] test_Whitelist_StartsEmpty() (gas: 20864)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.65ms (97.54µs CPU time)

Ran 4 tests for test/foundry/fuzz/GuardControllerFuzz.t.sol:GuardControllerFuzzTest
[PASS] testFuzz_ExecutionParams(bytes4,bytes,uint256) (runs: 256, : 24632, ~: 24541)
[PASS] testFuzz_Payment(uint256) (runs: 256, : 42553, ~: 42553)
[PASS] testFuzz_RegisterFunction(string,string,uint8[]) (runs: 256, : 37260, ~: 36126)
[PASS] testFuzz_TargetWhitelistExecutionParams(bytes4,address,bool) (runs: 256, : 13293, ~: 13293)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 914.44ms (81.59ms CPU time)

Ran 2 tests for test/foundry/security/Reentrancy.t.sol:ReentrancyTest
[PASS] test_ReentrancyProtection_OwnershipTransfer() (gas: 523871)
[PASS] test_ReentrancyProtection_StateMachinePrevents() (gas: 506159)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.51ms (471.63µs CPU time)

Ran 23 tests for test/foundry/fuzz/ComprehensiveStateMachineFuzz.t.sol:ComprehensiveStateMachineFuzzTest
[PASS] testFuzz_BlockTimestampManipulationLimited(bytes,uint256) (runs: 256, : 433408, ~: 422021)
[PASS] testFuzz_CatchBlockNotTriggeredByDeliberateOOG(bytes) (runs: 256, : 468524, ~: 458235)
[PASS] testFuzz_ConcurrentApprovalCancellationPrevented(bytes) (runs: 256, : 529154, ~: 518699)
[PASS] testFuzz_ERC20TokenReentrancyPrevented(uint256) (runs: 256, : 615327, ~: 615316)
[PASS] testFuzz_FunctionSchemaConsistency() (gas: 9539)
[PASS] testFuzz_GasLimitManipulationHandled(bytes,uint256) (runs: 256, : 499026, ~: 516177)
[PASS] testFuzz_InvalidStatusTransitionPrevented(bytes) (runs: 256, : 507244, ~: 496894)
[PASS] testFuzz_NoDelegatecallToUserControlledTarget() (gas: 375113)
[PASS] testFuzz_NoPartialStateOnRevert(bytes) (runs: 256, : 501060, ~: 490650)
[PASS] testFuzz_PaymentRecipientReentrancyPrevented(uint256) (runs: 256, : 383894, ~: 383883)
[PASS] testFuzz_PrematureApprovalPrevented(bytes,uint256) (runs: 256, : 432503, ~: 421159)
[PASS] testFuzz_ReleaseTimeAnchoredToRequestTime(bytes) (runs: 256, : 507100, ~: 496726)
[PASS] testFuzz_TargetContractRevertHandled(bytes) (runs: 256, : 514086, ~: 503573)
[PASS] testFuzz_TargetReentrancyPrevented(bytes) (runs: 256, : 489003, ~: 478409)
[PASS] testFuzz_TimeLockPeriodManipulationPrevented(uint256) (runs: 256, : 428599, ~: 428589)
[PASS] test_BlockTimestampManipulationLimited_AuthorizedContext() (gas: 378476)
[PASS] test_ConcurrentApprovalCancellationPrevented_AuthorizedContext() (gas: 417378)
[PASS] test_GasLimitManipulationHandled_AuthorizedContext() (gas: 411590)
[PASS] test_InsufficientBalanceHandled_AuthorizedContext() (gas: 480715)
[PASS] test_InvalidStatusTransitionPrevented_AuthorizedContext() (gas: 407084)
[PASS] test_PrematureApprovalPrevented_AuthorizedContext() (gas: 377353)
[PASS] test_TargetContractRevertHandled_AuthorizedContext() (gas: 446251)
[PASS] test_TargetReentrancyPrevented_AuthorizedContext() (gas: 419748)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 993.06ms (962.49ms CPU time)

Ran 19 tests for test/foundry/unit/BaseStateMachine.t.sol:BaseStateMachineTest
[PASS] test_CreateMetaTxParams_ValidParams() (gas: 15231)
[PASS] test_GetActiveRolePermissions_ReturnsPermissions() (gas: 331605)
[PASS] test_GetBroadcasters_ReturnsBroadcasterAddresses() (gas: 21127)
[PASS] test_GetPendingTransactions_ReturnsPendingOnly() (gas: 425750)
[PASS] test_GetRecovery_ReturnsRecoveryAddress() (gas: 18347)
[PASS] test_GetSignerNonce_ReturnsCorrectNonce() (gas: 28463)
[PASS] test_GetSupportedFunctions_ReturnsAllFunctions() (gas: 110384)
[PASS] test_GetSupportedOperationTypes_ReturnsAllTypes() (gas: 48599)
[PASS] test_GetSupportedRoles_ReturnsAllRoles() (gas: 29339)
[PASS] test_GetTimeLockPeriodSec_ReturnsCorrectPeriod() (gas: 8630)
[PASS] test_GetTransactionHistory_EmptyWhenNoTransactions() (gas: 17443)
[PASS] test_GetTransactionHistory_EmptyWhenRangeDoesNotOverlap() (gas: 423694)
[PASS] test_GetTransactionHistory_ReturnsRange() (gas: 904180)
[PASS] test_GetTransaction_ReturnsCorrectTransaction() (gas: 447854)
[PASS] test_HasRole_ReturnsCorrectPermissions() (gas: 55185)
[PASS] test_Initialized_ReturnsTrue() (gas: 10468)

```
[PASS] test_Owner_ReturnsOwnerAddress() (gas: 17913)
[PASS] test_SupportsInterface_ERC165() (gas: 6027)
[PASS] test_SupportsInterface_IBaseStateMachine() (gas: 7167)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 3.77ms (1.10ms CPU time)

Ran 4 tests for test/foundry/fuzz/EdgeCasesFuzz.t.sol:EdgeCasesFuzzTest
[PASS] testFuzz_EmptyBatchOperations() (gas: 464827)
[PASS] testFuzz_LargeBatchOperations(uint8) (runs: 256, : 6881001, ~: 4413462)
[PASS] testFuzz_MixedBatchOperations(uint8) (runs: 256, : 2246728, ~: 2008514)
[PASS] testFuzz_RoleNameEdgeCases(string) (runs: 256, : 861164, ~: 888602)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 781.93ms (775.39ms CPU time)

Ran 11 tests for test/foundry/unit/GuardController.t.sol:GuardControllerTest
[PASS] test_ApproveTimeLockExecution_AfterTimelock() (gas: 23387)
[PASS] test_CancelTimeLockExecution_Authorized() (gas: 24421)
[PASS] test_EmptyWhitelist_DeniesAllExecutions() (gas: 20535)
[PASS] test_ExecuteWithTimeLock_Revert_NonWhitelistedTarget() (gas: 23937)
[PASS] test_GetAllowedTargets_RequiresRole() (gas: 16446)
[PASS] test_GetAllowedTargets_ReturnsWhitelistedTargets() (gas: 20821)
[PASS] test_GuardConfigBatchExecutionParams_AddTargetToWhitelist() (gas: 11993)
[PASS] test_GuardConfigBatchExecutionParams_RemoveTargetFromWhitelist() (gas: 11900)
[PASS] test_GuardConfigBatchExecutionParams_Revert_ZeroAddress() (gas: 7936)
[PASS] test_SupportsInterface_ERC165() (gas: 6071)
[PASS] test_SupportsInterface_IGuardController() (gas: 5755)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.76ms (163.29µs CPU time)

Ran 5 tests for test/foundry/fuzz/StateMachineWorkflowFuzz.t.sol:StateMachineWorkflowFuzzTest
[PASS] testFuzz_CompleteTransactionLifecycle(address,bytes4,bytes,uint256) (runs: 256, : 25924, ~: 25710)
[PASS] testFuzz_ConcurrentTransactions(uint8) (runs: 256, : 43724, ~: 41143)
[PASS] testFuzz_PrematureApprovalFails(address,bytes4,bytes) (runs: 256, : 25538, ~: 25491)
[PASS] testFuzz_TransactionCancellation(address,bytes4,bytes) (runs: 256, : 25340, ~: 25293)
[PASS] testFuzz_TransactionStatusTransitions(address,bytes4) (runs: 256, : 24624, ~: 24534)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 79.40ms (76.53ms CPU time)

Ran 9 tests for test/foundry/fuzz/ComprehensiveInitializationFuzz.t.sol:ComprehensiveInitializationFuzzTest
[PASS] testFuzz_AllZeroAddressesPrevented() (gas: 4196552)
[PASS] testFuzz_MultipleInitializationPrevented(address,address,address) (runs: 256, : 20214483, ~: 20214483)
[PASS] testFuzz_MultipleInitializationWithSameParamsPrevented() (gas: 20197911)
[PASS] testFuzz_UninitializedStateExploitationPrevented() (gas: 15874)
[PASS] testFuzz_ValidInitializationSucceeds(address,address,address,uint256) (runs: 256, : 20201713, ~: 20201634)
[PASS] testFuzz_ZeroBroadcasterAddressPrevented() (gas: 4200593)
[PASS] testFuzz_ZeroOwnerAddressPrevented() (gas: 4200911)
[PASS] testFuzz_ZeroRecoveryAddressPrevented() (gas: 4200813)
[PASS] testFuzz_ZeroTimeLockPeriodPrevented() (gas: 4203460)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 1.47s (1.47s CPU time)

Ran 6 tests for test/foundry/fuzz/SecureOwnableFuzz.t.sol:SecureOwnableFuzzTest
[PASS] testFuzz_BroadcasterUpdate(address) (runs: 256, : 519397, ~: 519397)
[PASS] testFuzz_MetaTransaction(uint256,uint256) (runs: 256, : 19433, ~: 19433)
[PASS] testFuzz_OwnershipRequest_WithBroadcasterUpdatePending(address) (runs: 256, : 777230, ~: 777230)
[PASS] testFuzz_OwnershipTransfer(uint256) (runs: 256, : 20576999, ~: 20576999)
[PASS] testFuzz_RecoveryUpdate(address) (runs: 256, : 15612, ~: 15612)
[PASS] testFuzz_TimeLockUpdate(uint256) (runs: 256, : 8577, ~: 8577)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.07s (1.06s CPU time)

Ran 5 tests for test/foundry/fuzz/ComprehensiveCompositeFuzz.t.sol:ComprehensiveCompositeFuzzTest
[PASS] testFuzz_MultiStagePermissionEscalationPrevented(uint256,uint256,address,bytes4,bytes4) (runs: 256, : 2762220, ~: 2762751)
[PASS] testFuzz_PaymentUpdateExecutionCombination(address,address,uint256) (runs: 256, : 9324, ~: 9324)
[PASS] testFuzz_TimeLockMetaTransactionsGatedByPermissions(string) (runs: 256, : 400912, ~: 400912)
[PASS] test_BatchWithProtectedRoleModification() (gas: 1016750)
[PASS] test_NoncePredictionReplayPrevented() (gas: 829837)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 566.21ms (671.03ms CPU time)

Ran 6 tests for test/foundry/invariant/StateMachineInvariants.t.sol:StateMachineInvariantsTest
[PASS] invariant_NoZeroAddressInProtectedRoles() (runs: 256, calls: 3840, reverts: 3840)

-----+-----+-----+-----+-----+
| Contract          | Selector                                | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| AccountPatternTest | approveTimeLockExecution                | 157   | 157     | 0         |
+-----+-----+-----+-----+-----+
| AccountPatternTest | approveTimeLockExecutionWithMetaTx      | 203   | 203     | 0         |
+-----+-----+-----+-----+-----+
| AccountPatternTest | cancelTimeLockExecution                  | 147   | 147     | 0         |
+-----+-----+-----+-----+-----+
```

AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0
AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0
AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0
AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

[PASS] invariant_OwnerRoleSingleWallet() (runs: 256, calls: 3840, reverts: 3840)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	157	157	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	203	203	0
AccountPatternTest	cancelTimeLockExecution	147	147	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0
AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0

AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0
AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

[PASS] invariant_PendingTransactionsConsistency() (runs: 256, calls: 3840, reverts: 3840)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	157	157	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	203	203	0
AccountPatternTest	cancelTimeLockExecution	147	147	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0
AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0
AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0
AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

[PASS] invariant_TimelockPeriodPositive() (runs: 256, calls: 3840, reverts: 3840)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	157	157	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	203	203	0
AccountPatternTest	cancelTimeLockExecution	147	147	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0
AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0
AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0
AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

[PASS] invariant_TransactionCounterMonotonic() (runs: 256, calls: 3840, reverts: 3840)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	157	157	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	203	203	0
AccountPatternTest	cancelTimeLockExecution	147	147	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0

AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0
AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0
AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

[PASS] invariant_ValidStatusTransitions() (runs: 256, calls: 3840, reverts: 3840)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	157	157	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	203	203	0
AccountPatternTest	cancelTimeLockExecution	147	147	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	170	170	0
AccountPatternTest	executeGuardConfigBatch	166	166	0
AccountPatternTest	executeRoleConfigBatch	146	146	0
AccountPatternTest	executeWithPayment	159	159	0
AccountPatternTest	executeWithTimeLock	153	153	0
AccountPatternTest	guardConfigBatchRequestAndApprove	150	150	0
AccountPatternTest	initialize	181	181	0
AccountPatternTest	requestAndApproveExecution	154	154	0
AccountPatternTest	roleConfigBatchRequestAndApprove	176	176	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	171	171	0
AccountPatternTest	transferOwnershipCancellation	172	172	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	155	155	0
AccountPatternTest	transferOwnershipDelayedApproval	154	154	0
AccountPatternTest	transferOwnershipRequest	162	162	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	139	139	0

AccountPatternTest	updateBroadcasterCancellation	145	145	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	164	164	0
AccountPatternTest	updateBroadcasterDelayedApproval	173	173	0
AccountPatternTest	updateBroadcasterRequest	172	172	0
AccountPatternTest	updateRecoveryRequestAndApprove	117	117	0
AccountPatternTest	updateTimeLockRequestAndApprove	154	154	0

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.35s (1.35s CPU time)

Ran 2 tests for test/foundry/fuzz/ComprehensiveHookSystemFuzz.t.sol:ComprehensiveHookSystemFuzzTest

[PASS] testFuzz_UnauthorizedHookSettingPrevented() (gas: 168)

[PASS] testFuzz_ZeroAddressHookPrevented() (gas: 234)

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.60ms (9.00µs CPU time)

Ran 19 tests for test/foundry/fuzz/AuditDerivedAttackVectorsFuzz.t.sol:AuditDerivedAttackVectorsFuzzTest

[PASS] testFuzz_Finding11_UnregisteredSelectorRevertsResourceNotFound(bytes4) (runs: 256, : 27668, ~: 27668)

[PASS] testFuzz_Finding14_PendingTxAfterWhitelistDelist_ApproveReverts() (gas: 407454)

[PASS] testFuzz_Finding14_PendingTxAfterWhitelistDelist_CancelReverts() (gas: 410214)

[PASS] testFuzz_Finding1_HandlerContractMismatchRejected(address) (runs: 256, : 111624, ~: 111624)

[PASS] testFuzz_Finding1_HandlerMismatch_GuardSignedRoleSubmitted() (gas: 78425)

[PASS] testFuzz_Finding1_HandlerSelectorMismatchRejected() (gas: 81166)

[PASS] testFuzz_Finding22_EmptyTransactionHistoryReturnsEmptyArray() (gas: 20199252)

[PASS] testFuzz_Finding22_NonOverlappingRangeReturnsEmpty(uint256) (runs: 256, : 20198355, ~: 20198262)

[PASS] testFuzz_Finding24_TxIdIsSequentialAndPredictable() (gas: 1474337)

[PASS] testFuzz_Finding29_ZeroGasLimitAcceptedInRequest() (gas: 390842)

[PASS] testFuzz_Finding31_LargeTimeLockAccepted(uint256) (runs: 256, : 20197119, ~: 20197035)

[PASS] testFuzz_Finding32_GuardBatchRejectsNonZeroPayment(uint256) (runs: 256, : 77083, ~: 76999)

[PASS] testFuzz_Finding3_RBACBatchRejectsNativeOnlyNonZeroPayment(uint256) (runs: 256, : 77247, ~: 77163)

[PASS] testFuzz_Finding3_RBACBatchRejectsNonZeroPayment(address,uint256,address,uint256) (runs: 256, : 78119, ~: 78119)

[PASS] testFuzz_Finding5_MaxResultPreviewBytesIs32KiB() (gas: 294)

[PASS] testFuzz_Finding6_NonWhitelistedERC20Rejected(address) (runs: 256, : 141067, ~: 141067)

[PASS] testFuzz_Finding6_NonWhitelistedRecipientRejected(address) (runs: 256, : 57186, ~: 57186)

[PASS] testFuzz_Finding7_OwnershipTransferSnapshotsCurrentRecovery() (gas: 438846)

[PASS] testFuzz_Finding8_RecoveryUpdateNotBlockedByPendingOwnership(address) (runs: 256, : 929715, ~: 929703)

Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 2.00s (1.99s CPU time)

Ran 5 tests for test/foundry/invariant/RoleInvariants.t.sol:RoleInvariantsTest

[PASS] invariant_FunctionPermissionConsistency() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0

AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_ProtectedRolesImmutable() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0

AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_ProtectedRolesNeverModifiedViaRuntimeRBAC() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0

AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_RoleHashConsistency() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0

MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0
MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

[PASS] invariant_RoleWalletLimits() (runs: 256, calls: 3840, reverts: 3194)

Contract	Selector	Calls	Reverts	Discards
AccountPatternTest	approveTimeLockExecution	114	114	0
AccountPatternTest	approveTimeLockExecutionWithMetaTx	113	113	0
AccountPatternTest	cancelTimeLockExecution	128	128	0
AccountPatternTest	cancelTimeLockExecutionWithMetaTx	121	121	0
AccountPatternTest	executeGuardConfigBatch	119	119	0
AccountPatternTest	executeRoleConfigBatch	132	132	0
AccountPatternTest	executeWithPayment	125	125	0
AccountPatternTest	executeWithTimeLock	99	99	0
AccountPatternTest	guardConfigBatchRequestAndApprove	119	119	0
AccountPatternTest	initialize	123	123	0
AccountPatternTest	requestAndApproveExecution	132	132	0
AccountPatternTest	roleConfigBatchRequestAndApprove	127	127	0
AccountPatternTest	transferOwnershipApprovalWithMetaTx	120	120	0
AccountPatternTest	transferOwnershipCancellation	123	123	0
AccountPatternTest	transferOwnershipCancellationWithMetaTx	143	143	0
AccountPatternTest	transferOwnershipDelayedApproval	123	123	0
AccountPatternTest	transferOwnershipRequest	114	114	0
AccountPatternTest	updateBroadcasterApprovalWithMetaTx	113	113	0
AccountPatternTest	updateBroadcasterCancellation	124	124	0
AccountPatternTest	updateBroadcasterCancellationWithMetaTx	135	135	0
AccountPatternTest	updateBroadcasterDelayedApproval	130	130	0
AccountPatternTest	updateBroadcasterRequest	150	150	0
AccountPatternTest	updateRecoveryRequestAndApprove	112	112	0
AccountPatternTest	updateTimeLockRequestAndApprove	109	109	0
MockERC20	approve	152	4	0
MockERC20	mint	128	4	0
MockERC20	transfer	128	126	0

MockERC20	transferFrom	116	112	0
MockEventForwarder	forwardEvent	137	0	0
MockTarget	execute	113	0	0
MockTarget	executeWithParams	118	0	0

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.10s (2.09s CPU time)

Ran 9 tests for test/foundry/fuzz/ComprehensivePaymentSecurityFuzz.t.sol:ComprehensivePaymentSecurityFuzzTest

[PASS] testFuzz_BalanceDrainPrevented(uint256,uint256) (runs: 256, : 1851842, ~: 1692982)
 [PASS] testFuzz_DoublePaymentPrevented(address,uint256) (runs: 256, : 512179, ~: 511954)
 [PASS] testFuzz_ERC20TokenAddressValidation(address,uint256) (runs: 256, : 139905, ~: 139692)
 [PASS] testFuzz_FeeOnTransferTokenHandling() (gas: 1177179)
 [PASS] testFuzz_PaymentAmountManipulationPrevented(uint256,uint256) (runs: 256, : 481736, ~: 479669)
 [PASS] testFuzz_PaymentRecipientUpdateAccessControl(address,address,uint256) (runs: 256, : 496684, ~: 496478)
 [PASS] testFuzz_PaymentUpdateTiming(address,address,uint256,uint256) (runs: 256, : 496641, ~: 496806)
 [PASS] test_ERC20TokenAddressZeroRejectedAtRequestTime(uint256) (runs: 256, : 127347, ~: 127108)
 [PASS] test_NativeRecipientZeroRejectedAtRequestTime(uint256) (runs: 256, : 46889, ~: 46650)

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 626.70ms (618.81ms CPU time)

Ran 10 tests for test/foundry/fuzz/ComprehensiveSecurityEdgeCasesFuzz.t.sol:ComprehensiveSecurityEdgeCasesFuzzTest

[PASS] testFuzz_BitmapOverflowPrevented(uint256) (runs: 256, ~: 1703)
 [PASS] testFuzz_CompositePaymentHookAttackPrevented(address,address,uint256) (runs: 256, : 487738, ~: 487527)
 [PASS] testFuzz_FrontRunningPaymentUpdateHandled(address,address,uint256) (runs: 256, : 466749, ~: 466538)
 [PASS] testFuzz_HandlerBitmapCombinationValidation(bytes4,bytes4) (runs: 256, : 1468022, ~: 1468022)
 [PASS] testFuzz_HookExecutionOrderConsistent(uint8) (runs: 256, : 1029417, ~: 809917)
 [PASS] testFuzz_HookInterfaceNonComplianceHandled() (gas: 164264)
 [PASS] testFuzz_HookReentrancyPrevented() (gas: 415723)
 [PASS] testFuzz_InvalidActionEnumValuesRejected(uint256) (runs: 256, : 2810, ~: 2825)
 [PASS] testFuzz_MultipleHooksGasExhaustionPrevented(uint8) (runs: 256, : 954472, ~: 755746)
 [PASS] testFuzz_PaymentUpdateRaceConditionPrevented(address,address,uint256) (runs: 256, : 471910, ~: 471699)

Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 532.22ms (527.21ms CPU time)

Ran 2 tests for test/foundry/fuzz/ComprehensiveEventForwardingFuzz.t.sol:ComprehensiveEventForwardingFuzzTest

[PASS] testFuzz_GasIntensiveEventForwarderHandled(address,uint256,bytes4,bytes) (runs: 256, : 20218011, ~: 20218007)
 [PASS] testFuzz_MaliciousEventForwarderIsolated(address,uint256,bytes4,bytes) (runs: 256, : 20218517, ~: 20218513)

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.26s (1.62s CPU time)

Ran 3 tests for test/foundry/fuzz/RBACPermissionFuzz.t.sol:RBACPermissionFuzzTest

[PASS] testFuzz_CannotAddDuplicateWallet(string,address) (runs: 256, : 2052004, ~: 2052015)
 [PASS] testFuzz_RoleWalletLimitEnforced(string,uint256,address[]) (runs: 256, : 9224139, ~: 3338765)
 [PASS] testFuzz_UnauthorizedRoleCannotExecute(address,bytes32,address) (runs: 256, : 22013, ~: 22013)

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.46s (3.46s CPU time)

Ran 13 tests for test/foundry/fuzz/ComprehensiveInputValidationFuzz.t.sol:ComprehensiveInputValidationFuzzTest

[PASS] testFuzz_ArrayIndexOutOfBoundsPrevented(bytes32,uint256) (runs: 256, : 14995, ~: 14995)
 [PASS] testFuzz_ArrayLengthManipulationHandled(uint256) (runs: 256, : 19088918, ~: 18136672)
 [PASS] testFuzz_ArrayLengthMismatchPrevented(uint256,uint256) (runs: 256, : 2003, ~: 2032)
 [PASS] testFuzz_EmptyArrayHandled() (gas: 465276)
 [PASS] testFuzz_FunctionSignatureValidation(string,bytes4) (runs: 256, : 5535, ~: 5541)
 [PASS] testFuzz_HandlerSelectorValidation(bytes4,bytes4) (runs: 256, : 5554, ~: 5554)
 [PASS] testFuzz_MaxWalletsValidation(uint256) (runs: 256, : 11716, ~: 514)
 [PASS] testFuzz_RoleNameLengthHandled(string) (runs: 256, : 854585, ~: 881853)
 [PASS] testFuzz_TimeLockPeriodBounds(uint256) (runs: 256, : 1049, ~: 1045)
 [PASS] testFuzz_ZeroAddressInRoleAssignment(string) (runs: 256, : 1342792, ~: 1342792)
 [PASS] testFuzz_ZeroAddressInjectionPrevented(bytes4,bytes) (runs: 256, : 23406, ~: 23407)
 [PASS] testFuzz_ZeroFunctionSelectorPrevented(address,bytes) (runs: 256, : 22601, ~: 22584)
 [PASS] testFuzz_ZeroOperationTypePrevented(address,bytes4,bytes) (runs: 256, : 22744, ~: 22743)

Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 15.84s (1.35s CPU time)

Ran 14 tests for test/foundry/fuzz/ComprehensiveAccessControlFuzz.t.sol:ComprehensiveAccessControlFuzzTest

[PASS] testFuzz_BatchOperationAtomicity(string,address) (runs: 256, : 1053924, ~: 1053912)
 [PASS] testFuzz_BatchWithMultipleProtectedRoleAttempts(address,address) (runs: 256, : 985545, ~: 985558)
 [PASS] testFuzz_CannotAddDuplicateWallet(string,address) (runs: 256, : 2050631, ~: 2050642)
 [PASS] testFuzz_CannotAddWalletToProtectedRoleViaBatch(address,uint256) (runs: 256, : 704754, ~: 704767)
 [PASS] testFuzz_CannotCreateRoleWithProtectedRoleName(string) (runs: 256, : 1691, ~: 1562)
 [PASS] testFuzz_CannotRemoveProtectedRole(uint256) (runs: 256, : 680765, ~: 682039)
 [PASS] testFuzz_CannotRevokeLastWalletFromProtectedRole(uint256) (runs: 256, : 466680, ~: 701459)
 [PASS] testFuzz_ConflictingMetaTxPermissionsRejected(string,bytes4) (runs: 256, : 798656, ~: 798644)
 [PASS] testFuzz_EmptyPermissionBitmapRejected(string,bytes4) (runs: 256, : 798304, ~: 798292)
 [PASS] testFuzz_HandlerSelectorValidationPreventsEscalation(bytes4,bytes4) (runs: 256, : 5220, ~: 5220)
 [PASS] testFuzz_PermissionAccumulationAcrossRoles(string,string,address,bytes4) (runs: 256, : 9371, ~: 9371)


```
[PASS] testFuzz_RoleWalletLimitEnforced(string,uint256) (runs: 256, : 3333201, ~: 2514)
[PASS] testFuzz_SelfReferenceOnlyForExecutionSelectors(bytes4) (runs: 256, : 4090, ~: 4090)
[PASS] testFuzz_StateConsistentAfterRemoval(string) (runs: 256, : 85946, ~: 85950)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 33.31s (2.53s CPU time)

Ran 14 tests for test/foundry/fuzz/ComprehensiveMetaTransactionFuzz.t.sol:ComprehensiveMetaTransactionFuzzTest
[PASS] testFuzz_ChainIdInDomainSeparator(uint256,uint256) (runs: 256, : 43875, ~: 43875)
[PASS] testFuzz_ConcurrentNonceUsagePrevented() (gas: 1396198)
[PASS] testFuzz_CrossChainSignatureReplayPrevented(uint256) (runs: 256, : 49257, ~: 49257)
[PASS] testFuzz_DeadlineExtensionAllowed(uint256) (runs: 256, : 812396, ~: 812413)
[PASS] testFuzz_ExpiredMetaTransactionRejected(uint256) (runs: 256, : 48599, ~: 48607)
[PASS] testFuzz_GasPriceLimitEnforced(uint256,uint256) (runs: 256, : 653257, ~: 653389)
[PASS] testFuzz_InvalidSignatureLengthRejected(uint256) (runs: 256, : 653659, ~: 653674)
[PASS] testFuzz_InvalidSignatureRejected() (gas: 671812)
[PASS] testFuzz_MessageHashManipulationPrevented(uint256) (runs: 256, : 727228, ~: 727227)
[PASS] testFuzz_NonceConsumedEvenOnExecutionFailure() (gas: 1190715)
[PASS] testFuzz_NonceIncrementsBeforeExecution() (gas: 1406583)
[PASS] testFuzz_NonceReplayPrevented(uint256) (runs: 256, : 672936, ~: 672936)
[PASS] testFuzz_SignatureMalleabilityPrevented() (gas: 675829)
[PASS] testFuzz_StructHashCollisionResistance(uint256,uint256,uint256,uint256) (runs: 256, : 44088, ~: 44088)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 32.83s (686.38ms CPU time)

Ran 17 tests for test/foundry/fuzz/ComprehensiveGasExhaustionFuzz.t.sol:ComprehensiveGasExhaustionFuzzTest
[PASS] testFuzz_BatchFunctionRegistrationGasConsumption(uint8) (runs: 256, : 28402725, ~: 15361413)
[PASS] testFuzz_BatchRoleCreationGasConsumption(uint8) (runs: 256, : 21770438, ~: 11801845)
[PASS] testFuzz_BatchSizeLimitEnforced(uint16) (runs: 256, : 80801447, ~: 79480919)
[PASS] testFuzz_CompositeGasExhaustionScenario(uint16,uint8) (runs: 256, : 16058400, ~: 5108114)
[PASS] testFuzz_FunctionCountLimitEnforced() (gas: 84586)
[PASS] testFuzz_FunctionRemovalGasExhaustionWithManyRoles(uint16) (runs: 256, : 62520433, ~: 51787722)
[PASS] testFuzz_HandlerValidationGasConsumption(uint8) (runs: 256, : 1863, ~: 1849)
[PASS] testFuzz_HasAnyRoleGasConsumptionWithManyRoles(uint16) (runs: 256, : 57810506, ~: 47712257)
[PASS] testFuzz_HookCountLimitEnforced(uint8) (runs: 256, : 979, ~: 943)
[PASS] testFuzz_HookExecutionGasConsumptionWithManyHooks(uint8) (runs: 256, : 2209, ~: 2028)
[PASS] testFuzz_PermissionCheckGasConsumptionWithManyRoles(uint16) (runs: 256, : 88246224, ~: 72372303)
[PASS] testFuzz_PermissionCheckOptimizationBenefit(uint8,uint8) (runs: 256, : 117699381, ~: 63884731)
[PASS] testFuzz_RoleCountLimitEnforced() (gas: 945855308)
[PASS] testFuzz_RoleFunctionPermissionsGasConsumption(uint8) (runs: 256, : 874050, ~: 873869)
[PASS] testFuzz_TransactionHistoryGasConsumption(uint16) (runs: 256, : 34106, ~: 34304)
[PASS] testFuzz_ViewFunctionGasConsumptionWithManyFunctions(uint16) (runs: 256, : 195711, ~: 195940)
[PASS] testFuzz_ViewFunctionGasConsumptionWithManyRoles(uint16) (runs: 256, : 297366819, ~: 207700968)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 33.51s (80.94s CPU time)

Ran 40 test suites in 34.31s (137.57s CPU time): 336 tests passed, 0 failed, 0 skipped (336 total tests)
```


9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.